El contenedor `set` y los montículos

Claves sin repetir en orden, y el mayor siempre a mano

Carlos A Delgado S

Pontificia Universidad Javeriana Cali

Objetivos de aprendizaje

Al finalizar esta sesión, el estudiante podrá

- Usar `set` con `insert`, `erase`, `find`, `count`, `lower_bound` y `upper_bound`, dar el costo de cada operación y escoger entre `set`, `map` y `unordered_map` (OA1, OA3).
- Decidir si un árbol o un arreglo es un montículo de máximos y pasar de un árbol binario completo a su arreglo y de vuelta (OA2).

Objetivos de aprendizaje

Al finalizar esta sesión, el estudiante podrá también

- Seguir a mano insertar y extraer el máximo en un montículo, con el árbol y el arreglo en cada paso, y dar su costo $O(\log n)$ (OA1, OA2).
- Implementar un montículo de máximos en C++ sobre un `vector` y resolver problemas con `set` (OA3, OA5).

Contenido

1 Repaso: el árbol binario de búsqueda

2 El contenedor `set`

3 Montículos

4 Ejercicios

5 Cierre

Contenido

1 Repaso: el árbol binario de búsqueda

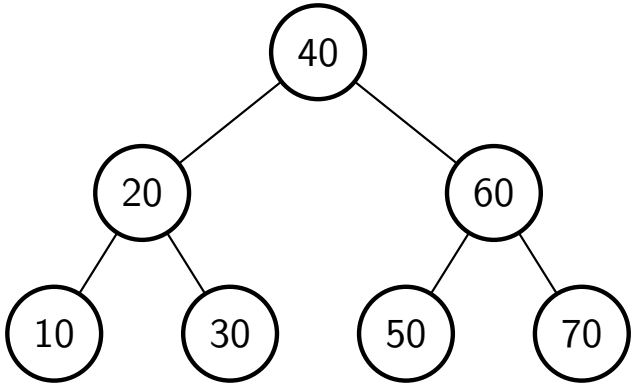
2 El contenedor `set`

3 Montículos

4 Ejercicios

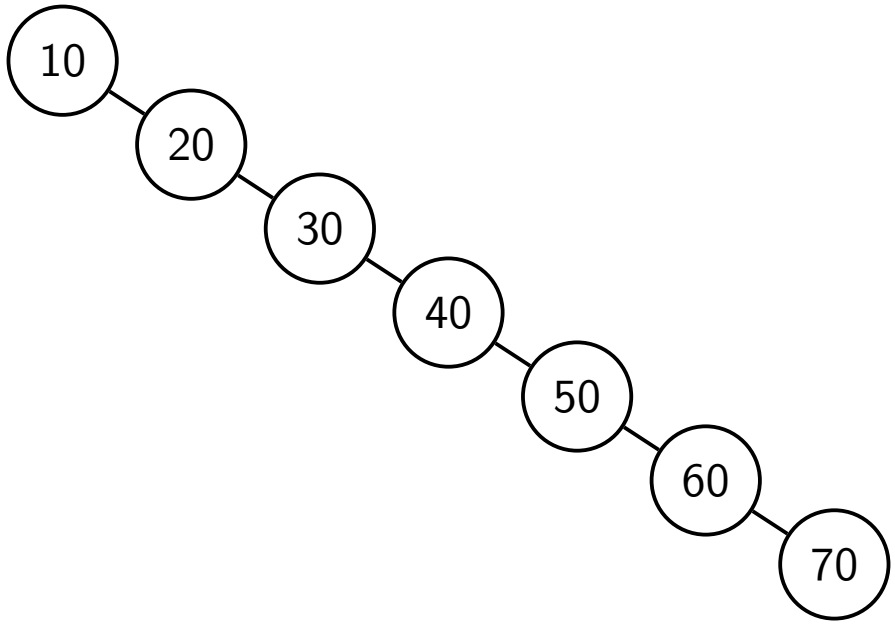
5 Cierre

Lo que quedó del árbol binario de búsqueda



- A la izquierda de cada nodo, claves menores; a la derecha, mayores.
- Buscar, insertar y borrar bajan por un solo camino: $O(h)$.
- Con n nodos, $\lfloor \log_2 n \rfloor \leq h \leq n - 1$.
- El inorden sale ordenado.

Lo que quedó del árbol binario de búsqueda



La forma la decide el orden de llegada

Insertadas como 40 20 60 10 30 50 70, estas siete claves dan altura 2. Insertadas como 10 20 30 40 50 60 70, dan esta lista de altura 6.

Lo que hay dentro de `map` y `set`

Un ABB que la biblioteca mantiene balanceado

`map` y `set` guardan sus claves en un árbol binario de búsqueda. Al insertar y al borrar, la biblioteca reacomoda unos pocos nodos cerca del camino por donde bajó, de modo que la altura queda en $O(\log n)$ sin importar el orden de llegada.

Lo que hay dentro de `map` y `set`

- `find`, `count`, `insert` y `erase` bajan por un camino de largo $O(\log n)$: cuestan $O(\log n)$ también en el peor caso.
- El recorrido con el iterador sale en orden porque es el inorden del árbol.
- En el `map`, el nodo guarda la pareja y el árbol se ordena por la clave. En el `set`, el nodo guarda solo la clave.

El ABB del curso frente a la biblioteca

	ABB del curso	`map`, `set`	`unordered_map`
buscar, insertar, borrar	$O(h)$: de $\Theta(\log n)$ a $\Theta(n)$	$O(\log n)$, también en el peor caso	$O(1)$ en promedio; $\Theta(n)$ en el peor caso
recorrer en orden de clave	$\Theta(n)$	$\Theta(n)$	copiar y ordenar: $\Theta(n \log n)$

El ABB del curso frente a la biblioteca

	ABB del curso	`map`, `set`	`unordered_map`
la primera clave $\geq x$	$O(h)$	$O(\log n)$ con `lower_bound`	no la tiene
lo que pide de la clave	<	<	una función hash y ==

El ABB del curso sirve para entender lo que hay dentro, pero no para datos que pueden llegar en orden.

Contenido

1 Repaso: el árbol binario de búsqueda

2 El contenedor `set`

3 Montículos

4 Ejercicios

5 Cierre

Solo las claves

Pregunta

Una lista de asistencia trae el código de cada estudiante cada vez que entra al aula: `42 17 88 17 5 63 42 30`. ¿Quiénes vinieron, de menor a mayor y sin repetir? Con un `map`, ¿qué valor se guardaría?

Solo las claves

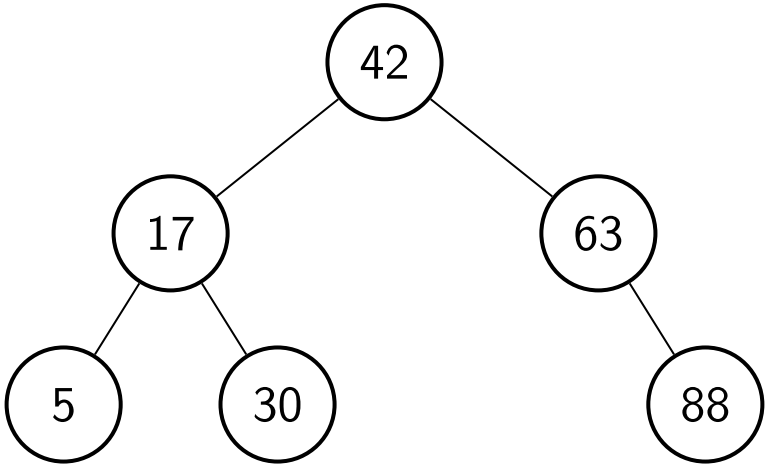
Respuesta

Vinieron `5 17 30 42 63 88`. Con un `map` habría que inventar un valor que nadie consulta. Lo que hace falta es un conjunto: claves sin repetir, en orden, sin nada al lado.

El contenedor `set`

Definición

`set<T>` guarda claves distintas de tipo `T`, ordenadas con `<`. Por dentro es el mismo árbol del `map`, con un nodo por clave y sin valor. Se incluye con `#include <set>`.



set en C++

uso_set.cpp mete la lista de asistencia en un set, la imprime, la consulta y pregunta por sus bordes.

```cpp
int main() {
  set<int> s;
  int llegan[8] = {42, 17, 88, 17, 5, 63, 42, 30};
  int i = 0;
  while (i < 8) {
    s.insert(llegan[i]);      // los segundos 17 y 42 no entran
    i = i + 1;
  }
  imprimir(s);
  consultar(s);
  imprimir(s);
  bordes(s);
  return 0;
}
```

set en C++

```cpp
// recorre el set con su iterador: las claves salen de menor a
↪  mayor
void imprimir(set<int> &s) {
  set<int>::iterator it = s.begin();
  while (it != s.end()) {
    printf(" %d", *it);
    ++it;
  }
  printf("   size = %d\n", (int) s.size());
}
```

set en C++

```
// count, find y erase sobre la lista de asistencia
void consultar(set<int> &s) {
  printf("count(17) = %d, count(18) = %d\n", (int)
  ↪  s.count(17),
        (int) s.count(18));
  printf("find(63) == end(): %d\n", (int) (s.find(63) ==
  ↪  s.end()));
  printf("erase(17) = %d\n", (int) s.erase(17));
  printf("erase(17) = %d\n", (int) s.erase(17));
}
```

set en C++

```
// la menor, la mayor y las primeras claves >= x y > x
void bordes(set<int> &s) {
  printf("menor: %d\n", *s.begin());
  set<int>::iterator ult = s.end();
  --ult;                          // end() queda despues del mayor
  printf("mayor: %d\n", *ult);
  printf("lower_bound(42) = %d\n", *s.lower_bound(42));
  printf("upper_bound(42) = %d\n", *s.upper_bound(42));
  printf("lower_bound(50) = %d\n", *s.lower_bound(50));
  printf("lower_bound(90) == end(): %d\n",
         (int) (s.lower_bound(90) == s.end()));
}
```

La salida de `uso_set.cpp`

Pregunta

Antes de ver la salida: ¿qué imprime el primer `imprimir`? ¿Cuánto vale `size()`? ¿Qué devuelve el segundo `erase(17)`?

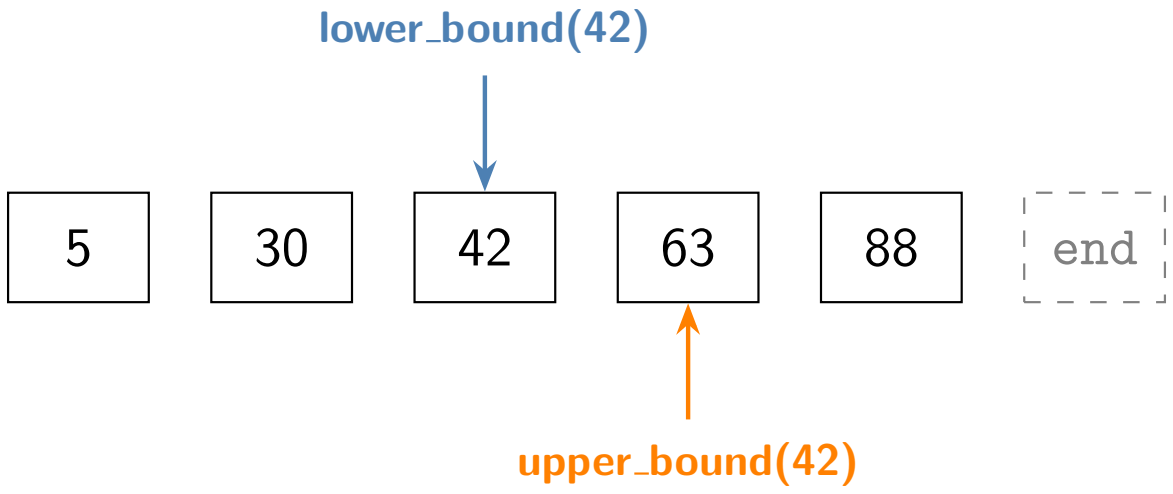
La salida de `uso_set.cpp`

```
 5 17 30 42 63 88   size = 6
count(17) = 1, count(18) = 0
find(63) == end(): 0
erase(17) = 1
erase(17) = 0
 5 30 42 63 88   size = 5
menor: 5
mayor: 88
lower_bound(42) = 42
upper_bound(42) = 63
lower_bound(50) = 63
lower_bound(90) == end(): 1
```

- El segundo 17 y el segundo 42 no entran: `insert` busca la clave y, si ya está, no hace nada.
- `erase(k)` devuelve cuántas claves quitó: 1 la primera vez, 0 la segunda.
- `end()` queda después del mayor. Retrocederlo una vez da el mayor.

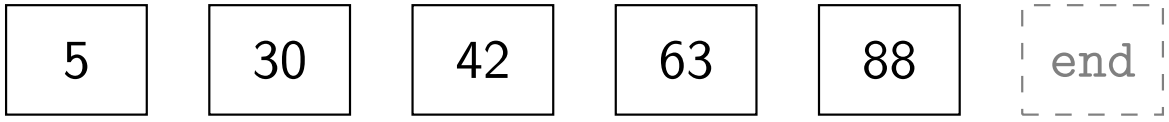
lower_bound y upper_bound

Con `s = {5, 30, 42, 63, 88}`:



- `lower_bound(x)`: iterador a la primera clave $\geq x$.
- `upper_bound(x)`: iterador a la primera clave $> x$.
- Los dos bajan por el árbol guardando el mejor candidato: $O(\log n)$.

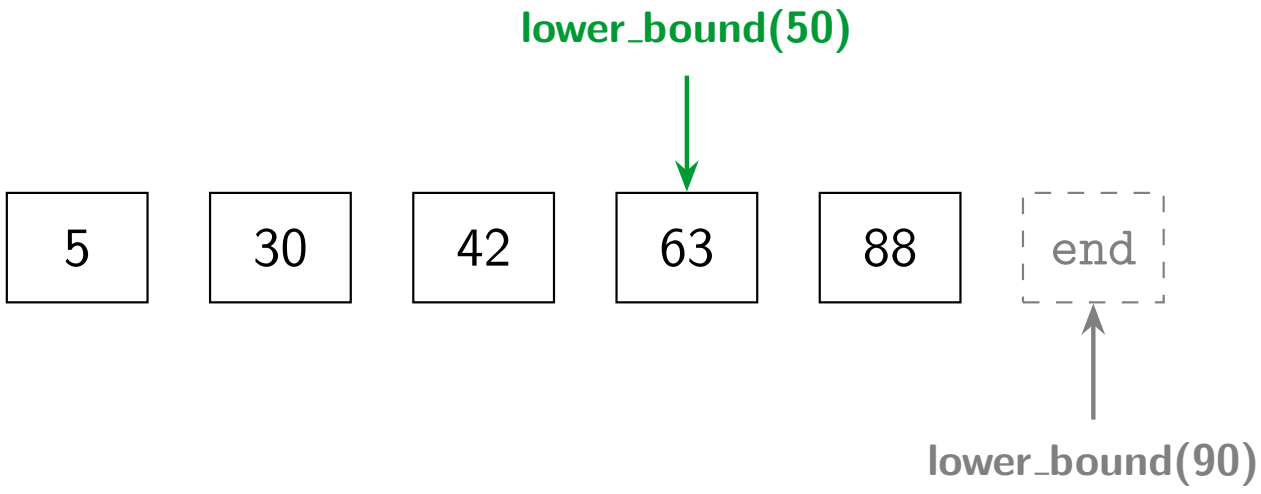
`lower_bound` y `upper_bound`



Pregunta

En el mismo `set`, ¿a qué apunta `lower_bound(50)`? ¿Y `lower_bound(90)`?

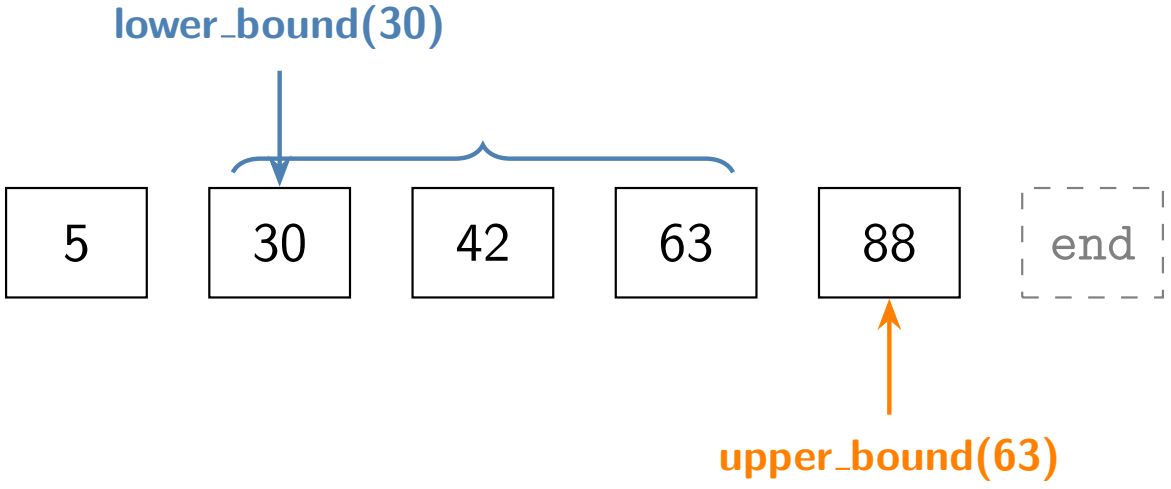
lower_bound y upper_bound



Respuesta

`lower_bound(50)` apunta al 63: el 50 no está y la primera clave mayor es el 63. Para el 90 no hay ninguna clave ≥ 90, y devuelve `end()`.

`lower_bound` y `upper_bound`



Las claves de $[a, b]$ van desde `lower_bound(a)` hasta la anterior a `upper_bound(b)`. Con $[30, 63]$ son el 30, el 42 y el 63.

`lower_bound` y `upper_bound`

No sirve el de `<algorithm>`

`lower_bound(s.begin(), s.end(), x)` compila, pero sobre un `set` avanza el iterador de uno en uno: $\Theta(n)$. Se usa el del contenedor, `s.lower_bound(x)`, que baja por el árbol en $O(\log n)$.

Lo que cuesta cada operación de `set`

Operación	Tiempo	Por qué
`insert(k)`	$O(\log n)$	se baja por un camino de largo $O(\log n)$ y se reacomodan pocos nodos cerca de él
`erase(k)`	$O(\log n)$	se busca, se quita y se reacomodan pocos nodos
`find(k)`, `count(k)`	$O(\log n)$	un camino desde la raíz
`lower_bound`, `upper_bound`	$O(\log n)$	un camino desde la raíz, guardando el mejor candidato
`size()`, `empty()`	$\Theta(1)$	el `set` lleva la cuenta

Lo que cuesta cada operación de `set`

Operación	Tiempo	Por qué
`begin()`	$\Theta(1)$	el `set` guarda dónde está el menor
`++it`, `--it`	$O(\log n)$	ir al sucesor puede subir o bajar toda la altura
recorrido completo	$\Theta(n)$	cada arista se baja una vez y se sube una vez
`erase(desde, hasta)` de m claves	$O(\log n + m)$	se quitan de a una, sin volver a buscar
espacio de la estructura	$\Theta(n)$	un nodo por clave, con punteros al padre y a los dos hijos

Escoger el contenedor

Pregunta

¿`set`, `map` o `unordered_map`?

1. Solo claves: si están, en orden, la menor, la siguiente a x.
2. Clave con valor, y la salida va en orden de clave.
3. Clave con valor, y solo se busca o se suma por clave.
4. Cuántas veces aparece cada clave.

Escoger el contenedor

Lo que se guarda y se pregunta	Contenedor
1. solo claves; si están, en orden, la menor, la siguiente a x	`set`
2. clave con valor; el orden aparece en la salida o se pregunta por rangos	`map`
3. clave con valor; solo se busca, se suma o se cuenta por clave	`unordered_map`
4. cuántas veces aparece cada clave	`map<T, int>` o `unordered_map<T, int>`, según haga falta el orden

Dos cuidados con el `set`

El `set` no guarda repetidos

Con `3 3 7`, el `set` queda con `3 7` y `size()` vale 2. Si las repeticiones importan, cada clave lleva su cuenta en un `map`.

Dos cuidados con el `set`

Una clave del `set` no se modifica

`*it` se puede leer pero no asignar: cambiar la clave en su sitio rompería el orden del árbol. Se borra la vieja y se inserta la nueva, $O(\log n)$ cada una.

Contenido

1 Repaso: el árbol binario de búsqueda

2 El contenedor `set`

3 Montículos

4 Ejercicios

5 Cierre

El pedido más urgente

Problema

Una central de domicilios recibe pedidos todo el día, cada uno con un número de urgencia: entre más alto, más urgente. Cada vez que un repartidor queda libre, se le asigna el pedido pendiente más urgente. Dos pedidos pueden tener la misma urgencia. Llegan cientos de miles de pedidos y de asignaciones.

¿Entendimos el problema?

Entradas y salidas

- Entradas: órdenes en secuencia, `llega u`, con la urgencia `u`, o `atender`.
- Salidas: para cada `atender`, la urgencia del pedido que se asigna.

¿Entendimos el problema?

Pregunta

Llegan 15, 40, 27 y 8; `atender`; llega 27; `atender`, `atender`, `atender`. ¿Qué se imprime? ¿Qué pedido queda pendiente?

Los pedidos, orden por orden

Orden	Pendientes después	Imprime
llegan 15, 40, 27, 8	15 40 27 8	

Los pedidos, orden por orden

Orden	Pendientes después	Imprime
llegan 15, 40, 27, 8	`15 40 27 8`	
`atender`	`15 27 8`	40

Los pedidos, orden por orden

Orden	Pendientes después	Imprime
llegan 15, 40, 27, 8	`15 40 27 8`	
`atender`	`15 27 8`	40
llega 27	`15 27 8 27`	

Los pedidos, orden por orden

Orden	Pendientes después	Imprime
llegan 15, 40, 27, 8	`15 40 27 8`	
`atender`	`15 27 8`	40
llega 27	`15 27 8 27`	
`atender`	`15 8 27`	27

Los pedidos, orden por orden

Orden	Pendientes después	Imprime
llegan 15, 40, 27, 8	`15 40 27 8`	
`atender`	`15 27 8`	40
llega 27	`15 27 8 27`	
`atender`	`15 8 27`	27
`atender`	`15 8`	27

Los pedidos, orden por orden

Orden	Pendientes después	Imprime
llegan 15, 40, 27, 8	`15 40 27 8`	
`atender`	`15 27 8`	40
llega 27	`15 27 8 27`	
`atender`	`15 8 27`	27
`atender`	`15 8`	27
`atender`	`8`	15

Respuesta

Se imprime `40 27 27 15` y queda el 8. Los dos 27 se atienden los dos: la urgencia repetida es otro pedido, no el mismo.

Primeras ideas

Pregunta

Con lo que ya se vio: un arreglo sin orden, un arreglo ordenado o un `set`. ¿Cuánto cuesta `llega` y cuánto `atender` con cada uno?

Primeras ideas

Estructura	`llega`	`atender`	Por qué
arreglo sin orden	$\Theta(1)$	$\Theta(n)$	se agrega al final; para atender se recorre buscando el mayor
arreglo ordenado de menor a mayor	$\Theta(n)$	$\Theta(1)$	al llegar se corren los mayores; el más urgente es el último
`set`	$O(\log n)$	$O(\log n)$	el mayor es el anterior a `end()`, y se borra
lo que se quiere	$O(\log n)$	$O(\log n)$	y que acepte urgencias repetidas

Primeras ideas

El `set` pierde un pedido

Con los pedidos del ejemplo, el segundo 27 no entra: el `set` no guarda repetidos, y uno de los dos pedidos nunca se atiende.

Basta conocer el mayor

Lo que de verdad se pregunta

Nadie pide la lista ordenada: solo cuál es el mayor. Basta un orden parcial, donde cada pedido sea más urgente o igual que los que dependen de él, y el mayor quede arriba.

Basta conocer el mayor

- Un árbol donde cada nodo es mayor o igual que sus hijos tiene el mayor en la raíz.
- Si además el árbol no tiene huecos, cabe en un arreglo sin punteros, y su altura es $\lfloor \log_2 n \rfloor$.

Esas dos ideas forman el montículo (*heap*; CLRS, cap. 6; Thareja, cap. 12).

Árbol binario completo

Definición (CLRS, §6.1; Thareja, §12.1)

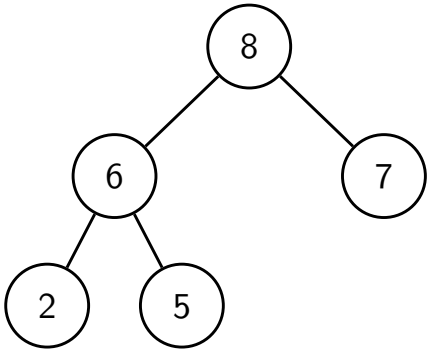
Un árbol binario es **completo** cuando todos sus niveles están llenos, salvo quizá el último, y el último se llena de izquierda a derecha, sin huecos.

Árbol binario completo

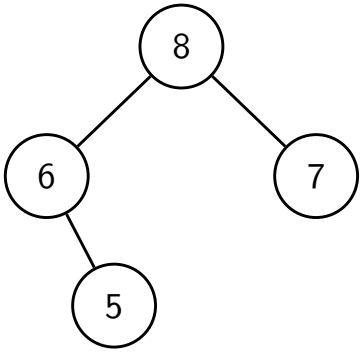
Pregunta

¿Cuáles de estos cuatro árboles son completos?

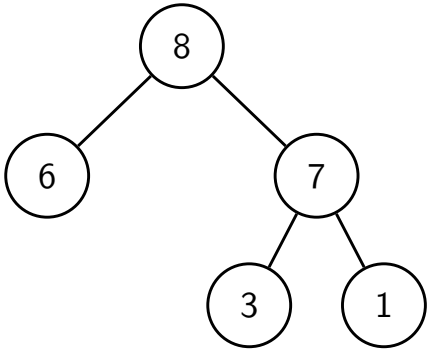
A



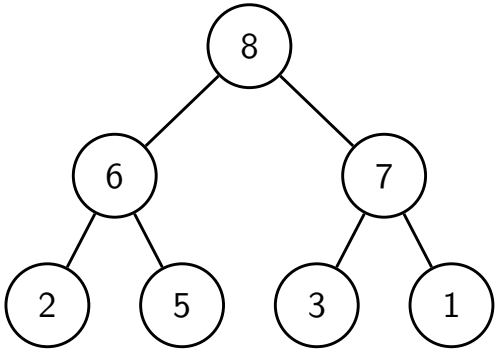
B



C



D

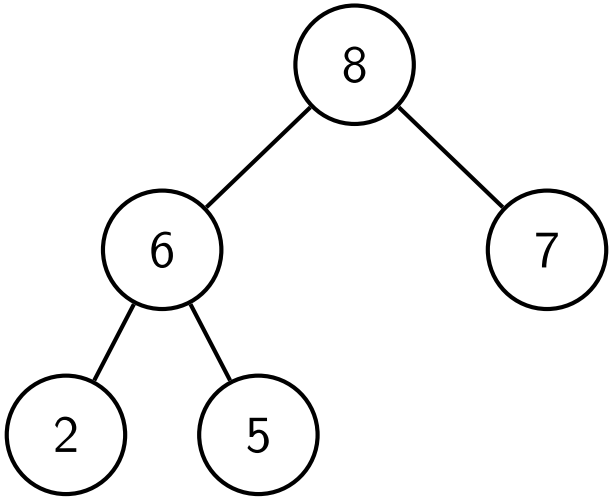


Árbol binario completo

Respuesta

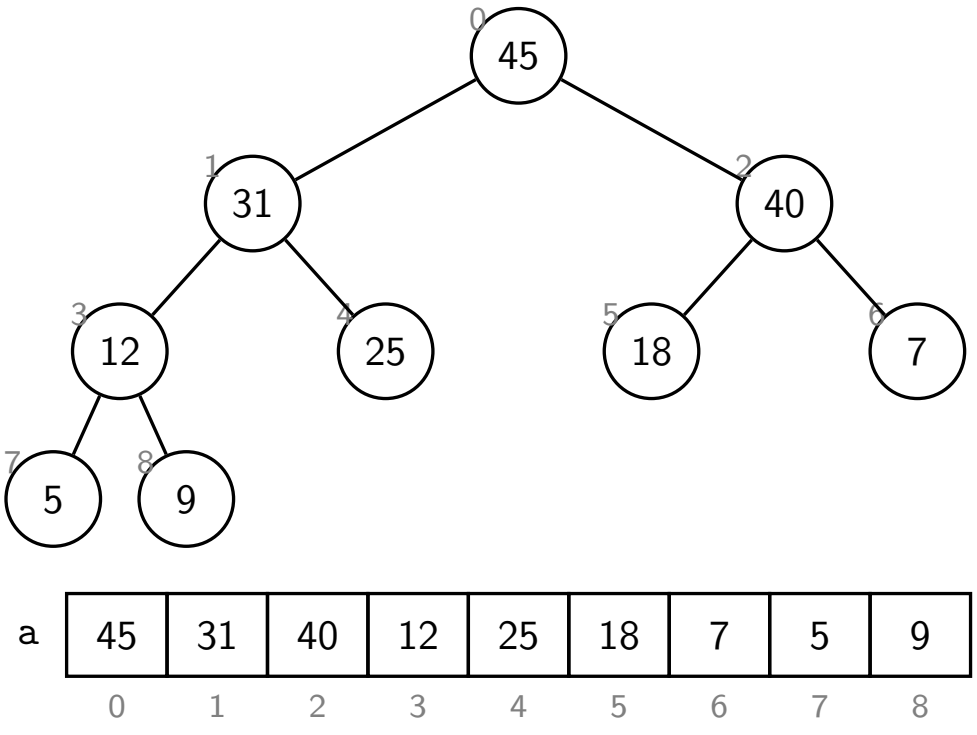
- **A** sí: el último nivel tiene dos nodos, pegados a la izquierda.
- **B** no: al último nivel le falta el primer hijo del 6 y tiene el segundo.
- **C** no: el último nivel empieza bajo el 7 y deja vacío el lugar bajo el 6.
- **D** sí: todos los niveles están llenos.

Árbol binario completo



En un árbol completo, la forma depende solo de n: con 5 nodos siempre es la de A. Por eso los nodos se pueden numerar por niveles, de izquierda a derecha, y guardar en un arreglo.

El árbol completo en un arreglo



El número gris es la casilla del arreglo. La raíz va en `a[0]`, sus hijos en `a[1]` y `a[2]`, los nietos en `a[3]` a `a[6]`, y así por niveles.

El árbol completo en un arreglo

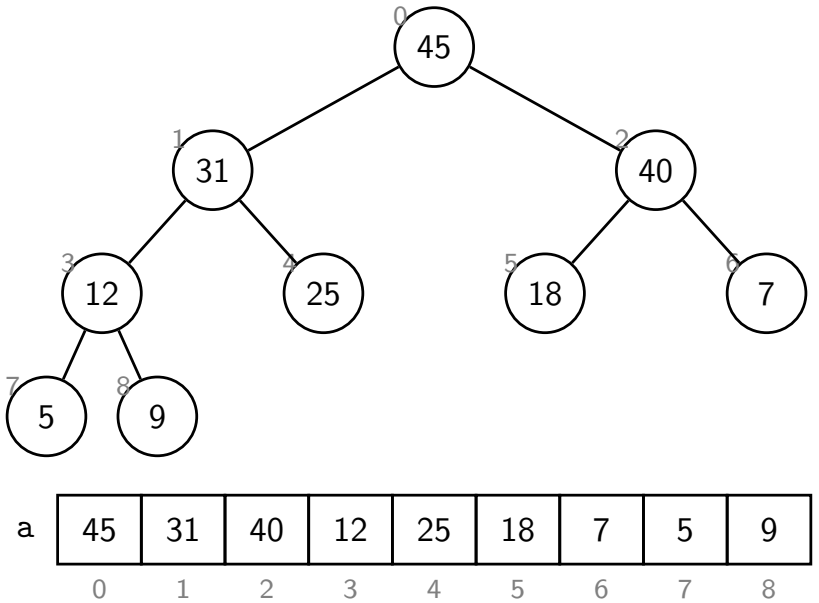
Padre e hijos con índices desde 0

Para la casilla i:

$$padre(i) = \left\lfloor \frac{i-1}{2} \right\rfloor \qquad izq(i) = 2i + 1 \qquad der(i) = 2i + 2$$

En C++, `(i - 1) / 2` con enteros ya redondea hacia abajo para $i \geq 1$.

El árbol completo en un arreglo



Con $n = 9$

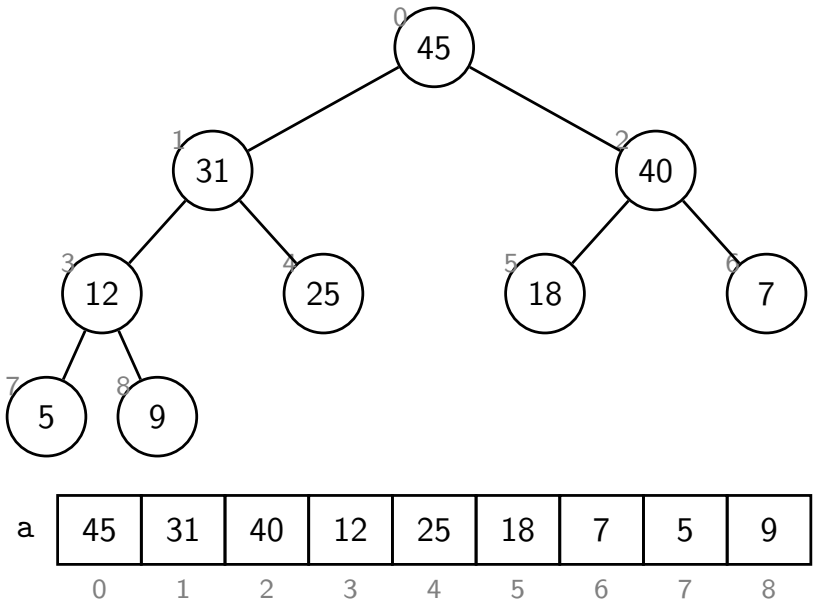
- La casilla 1 (el 31) tiene hijos en 3 y 4, y su padre es la 0.
- La casilla 4 (el 25) tendría hijos en 9 y 10, fuera del arreglo: es una hoja.
- La casilla 8 (el 9) tiene padre $\lfloor 7/2 \rfloor = 3$.

El árbol completo en un arreglo

CLRS y Thareja cuentan desde 1

Con la raíz en la casilla 1, el padre es $\lfloor i/2 \rfloor$ y los hijos $2i$ y $2i+1$. Aquí se cuenta desde 0, como en un `vector`, y por eso aparecen los $+1$ y $+2$.

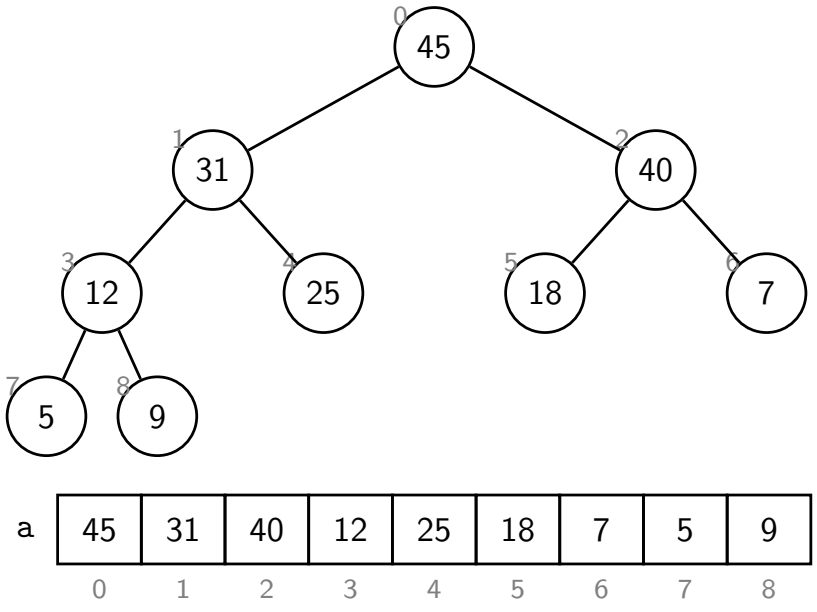
El árbol completo en un arreglo



Pregunta

¿Quiénes son los hijos del 40? ¿Quién es el padre del 7? Si entra una décima clave, ¿en qué casilla queda y quién es su padre?

El árbol completo en un arreglo



Respuesta

- El 40 está en la casilla 2: sus hijos son el 18 y el 7.
- El 7 está en la casilla 6: su padre es $\lfloor 5/2 \rfloor = 2$, el 40.
- La décima va en la casilla 9, con padre $\lfloor 8/2 \rfloor = 4$, el 25.

La altura de un árbol completo

Con 9 nodos, los niveles 0, 1 y 2 se llenan con 7 y los 2 que sobran quedan en el nivel 3: la altura es 3.

n	$h = \lfloor \log_2 n \rfloor$	por qué
9	3	$2^3 = 8 \leq 9 < 16$
1 000	9	$512 \leq 1\,000 < 1\,024$
1 000 000	19	$524\,288 \leq 10^6 < 1\,048\,576$

La altura de un árbol completo

Un árbol completo de altura h tiene los niveles $0, \ldots, h-1$ llenos y al menos un nodo en el nivel h:

$$2^h = (1 + 2 + \cdots + 2^{h-1}) + 1 \;\leq\; n \;\leq\; 1 + 2 + \cdots + 2^h = 2^{h+1} - 1.$$

De $2^h \leq n < 2^{h+1}$, tomando $\log_2$, sale $h \leq \log_2 n < h + 1$, es decir, $h = \lfloor \log_2 n \rfloor$.

La altura de un árbol completo

La altura no depende del orden de llegada

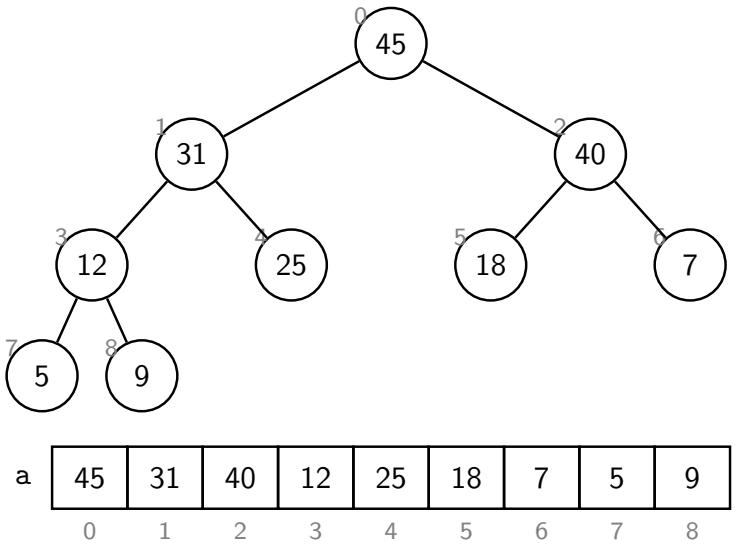
Un camino de la raíz a una hoja tiene a lo sumo $\lfloor \log_2 n \rfloor$ aristas, sin importar el orden en que llegaron las claves. Un ABB no da esa garantía: su forma depende del orden.

Montículo de máximos

Definición (CLRS, §6.1; Thareja, §12.1)

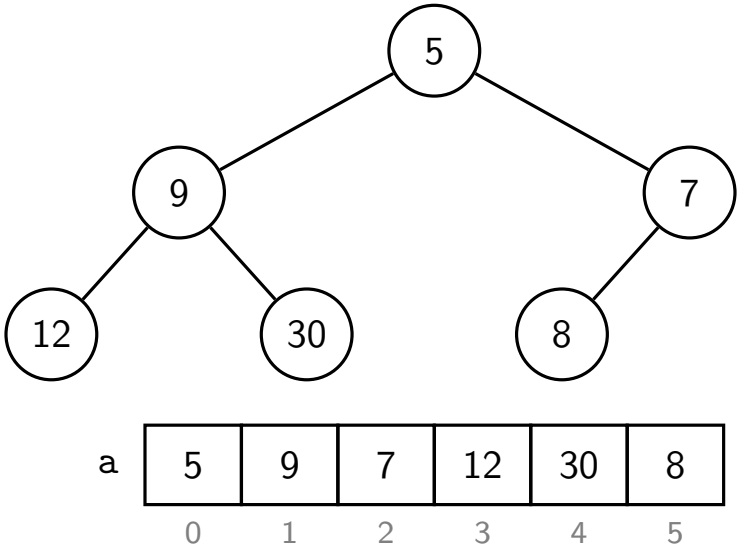
Un **montículo de máximos** es un árbol binario completo, guardado en un arreglo, en el que cada nodo es mayor o igual que sus hijos: para toda casilla $i \geq 1$, $a[padre(i)] \geq a[i]$.

Montículo de máximos



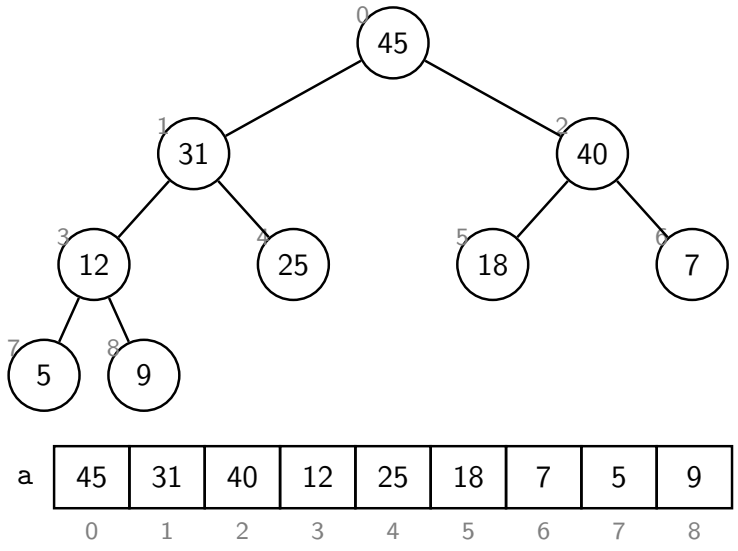
- Siguiendo cualquier camino hacia arriba, las claves no bajan: el mayor queda en la raíz, `a[0]`.
- Entre hermanos no hay orden: el 12 está a la izquierda del 40 y es menor que el 18, que está a la derecha.

Montículo de máximos



Un **montículo de mínimos** es lo mismo con $\leq$: cada nodo es menor o igual que sus hijos, y el menor queda en la raíz.

Montículo de máximos



No es un ABB

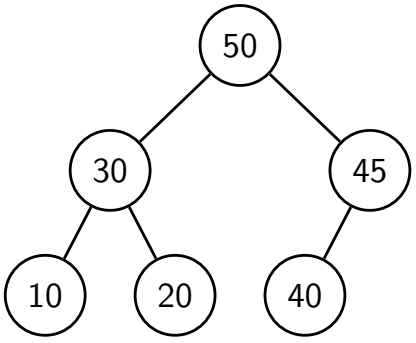
El inorden de este montículo es 5 12 9 31 25 45 18 40 7: no sale ordenado. Buscar una clave cualquiera no puede descartar un subárbol y cuesta $\Theta(n)$.

¿Cuáles son montículos de máximos?

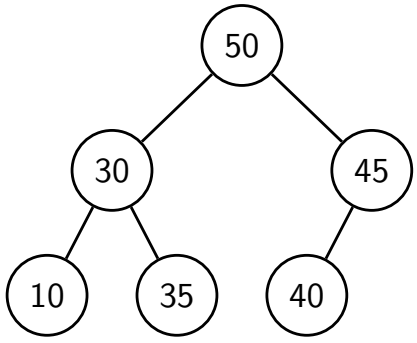
Pregunta

Para cada árbol, ¿es completo? ¿Cada nodo es mayor o igual que sus hijos?

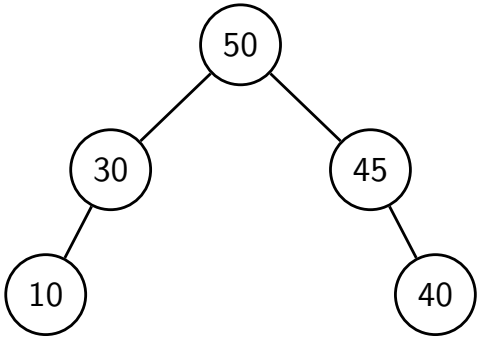
A



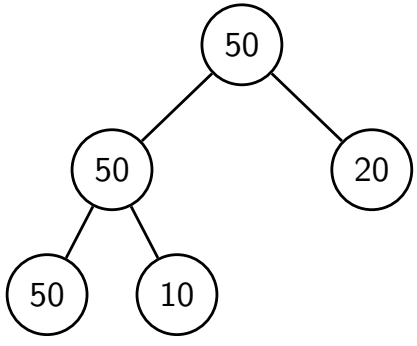
B



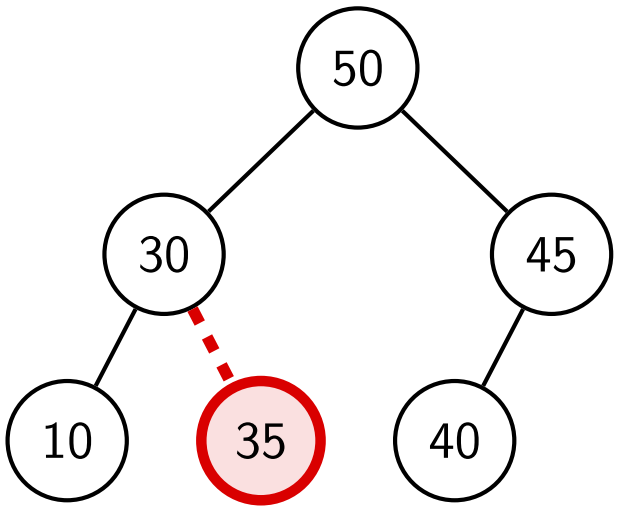
C



D



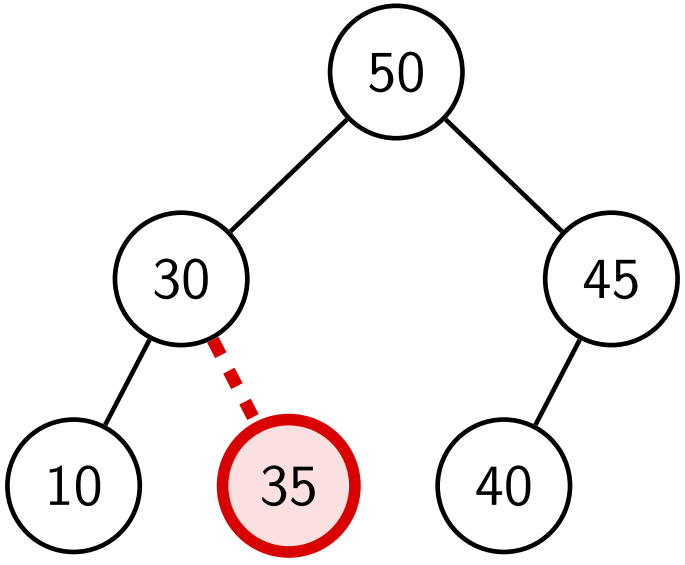
¿Cuáles son montículos de máximos?



Respuesta

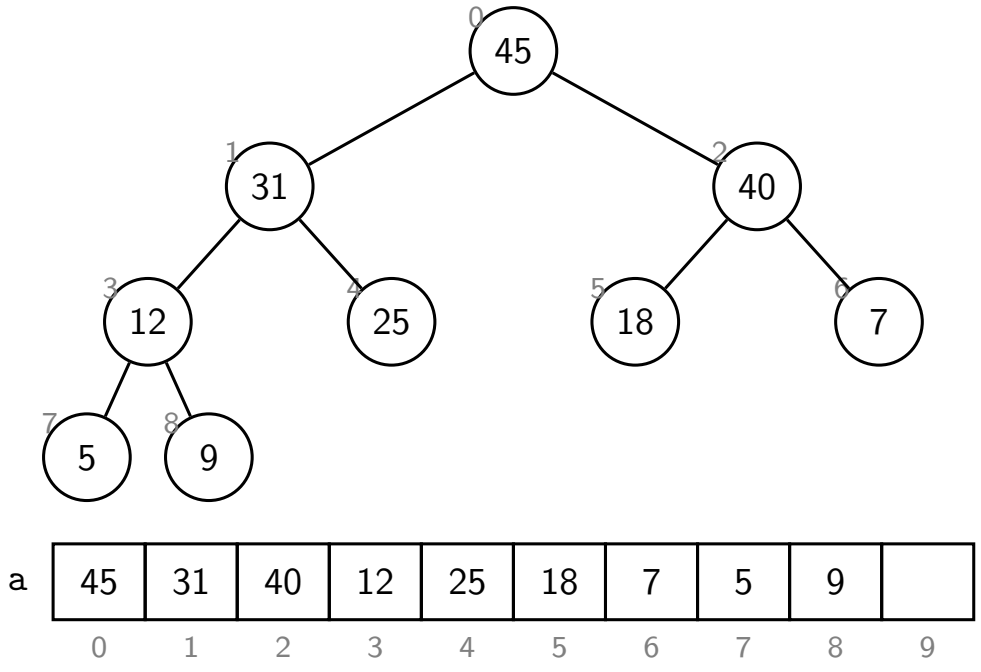
- **A** sí: completo, y cada nodo es mayor que sus hijos.
- **B** no: el 35 es hijo del 30; se compara con su padre, no con la raíz.
- **C** no: cumple el orden, pero no es completo.
- **D** sí: con iguales, *mayor o igual* se cumple.

¿Cuáles son montículos de máximos?



Revisar un arreglo cuesta $\Theta(n)$: cada casilla $i \geq 1$ se compara una vez con su padre. En B, la comparación que falla es la del 35 con el 30.

Insertar: dónde entra la clave



Pregunta

Llega el 42. ¿En qué casilla puede entrar sin que el árbol deje de ser completo? ¿Queda ahí un montículo?

Insertar: dónde entra la clave

Respuesta

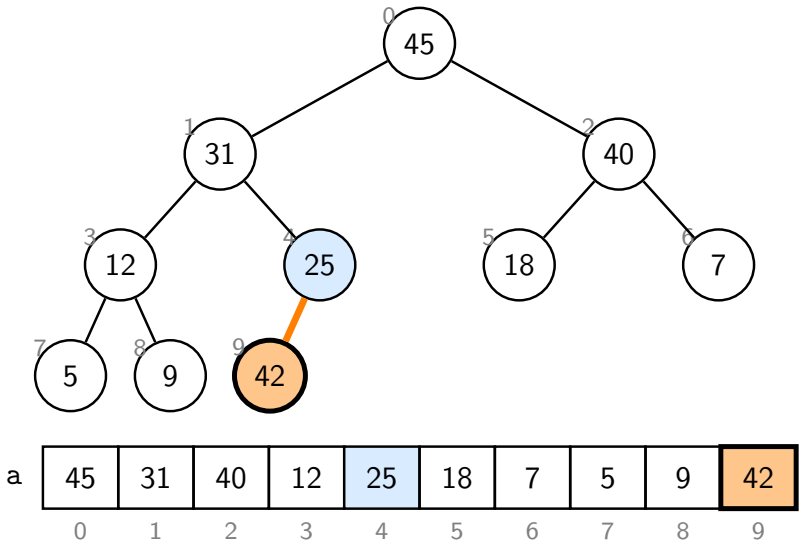
Solo en la casilla 9, la primera libre: es el siguiente lugar del último nivel, de izquierda a derecha. La forma queda completa, pero el 42 queda bajo el 25, que es menor.

Insertar: subir

Idea (CLRS, §6.5; Thareja, §12.1.1)

La clave nueva se compara con su padre. Si es mayor, se intercambian y se repite desde la casilla del padre. Se detiene en la raíz o cuando el padre es mayor o igual.

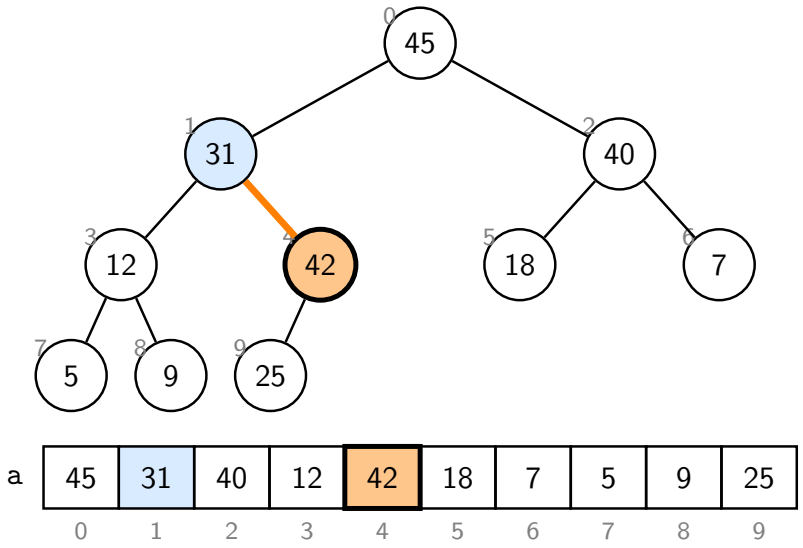
Insertar 42: traza



El 42 entra en a[9]. Su padre es a[4] = 25, y 42 > 25: se intercambian.

i	$padre(i)$	comparación	acción
9	4	$42 > 25$	intercambiar, $i = 4$

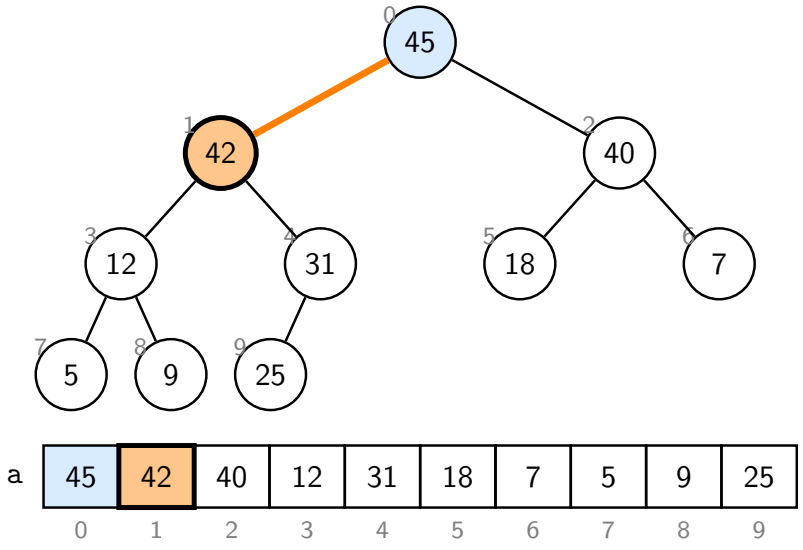
Insertar 42: traza



El 42 está en a[4]. Su padre es a[1] = 31, y 42 > 31: se intercambian.

i	$padre(i)$	comparación	acción
9	4	$42 > 25$	intercambiar, $i = 4$
4	1	$42 > 31$	intercambiar, $i = 1$

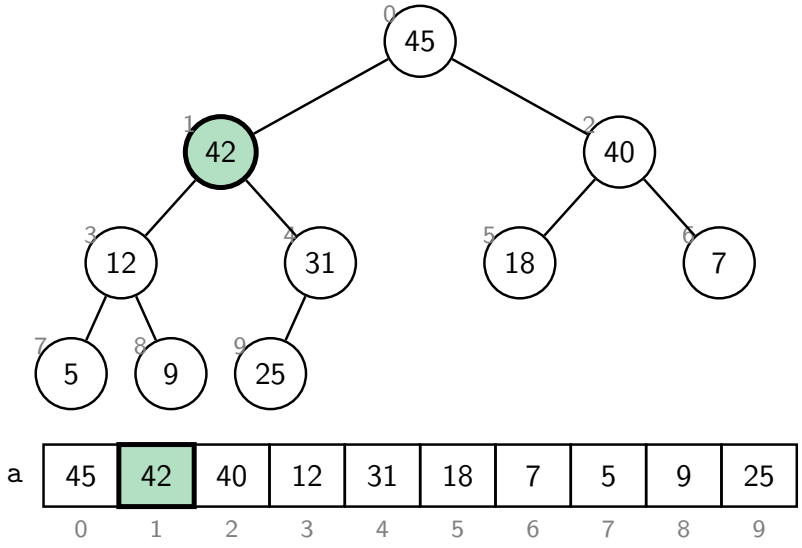
Insertar 42: traza



El 42 está en `a[1]`. Su padre es `a[0]` $= 45$, y $42 < 45$: se detiene.

i	$padre(i)$	comparación	acción
9	4	$42 > 25$	intercambiar, $i = 4$
4	1	$42 > 31$	intercambiar, $i = 1$
1	0	$42 < 45$	detenerse

Insertar 42: traza



Queda 45 42 40 12 31 18 7 5 9 25: el 42 subió dos niveles.

i	$padre(i)$	comparación	acción
9	4	$42 > 25$	intercambiar, $i = 4$
4	1	$42 > 31$	intercambiar, $i = 1$
1	0	$42 < 45$	detenerse

Insertar en C++

`monticulo.h` guarda las claves en un `vector<int> a`. `subir` e `intercambiar` son privados.

```
class Monticulo {
private:
  vector<int> a;

  void intercambiar(int i, int j) {
    int aux = a[i];
    a[i] = a[j];
    a[j] = aux;
  }
```

Insertar en C++

```
// Sube la clave de i mientras sea mayor que la de su padre.
void subir(int i) {
  while (i > 0 && a[(i - 1) / 2] < a[i]) {
    intercambiar(i, (i - 1) / 2);
    i = (i - 1) / 2;
  }
}
```

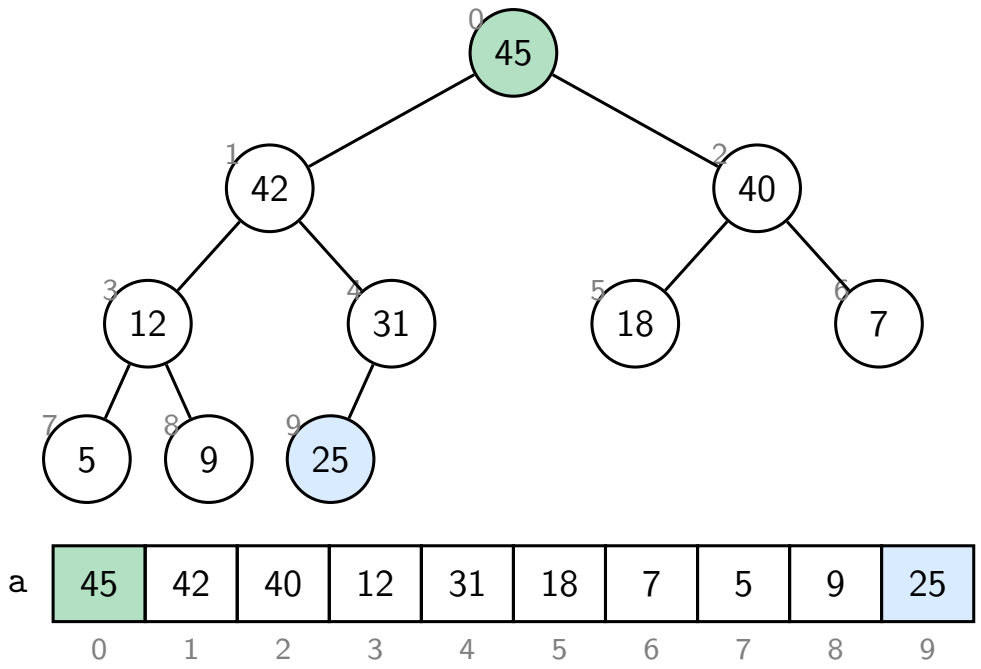
- La condición del `while` se detiene en la raíz (`i > 0`) o cuando el padre ya no es menor.
- `(i - 1) / 2` es la división entera: el padre de 9 es 4 y el padre de 4 es 1.

Insertar en C++

```
public:
  // La clave entra en la primera casilla libre y sube.
  void insertar(int k) {
    a.push_back(k);
    subir((int) a.size() - 1);
  }
```

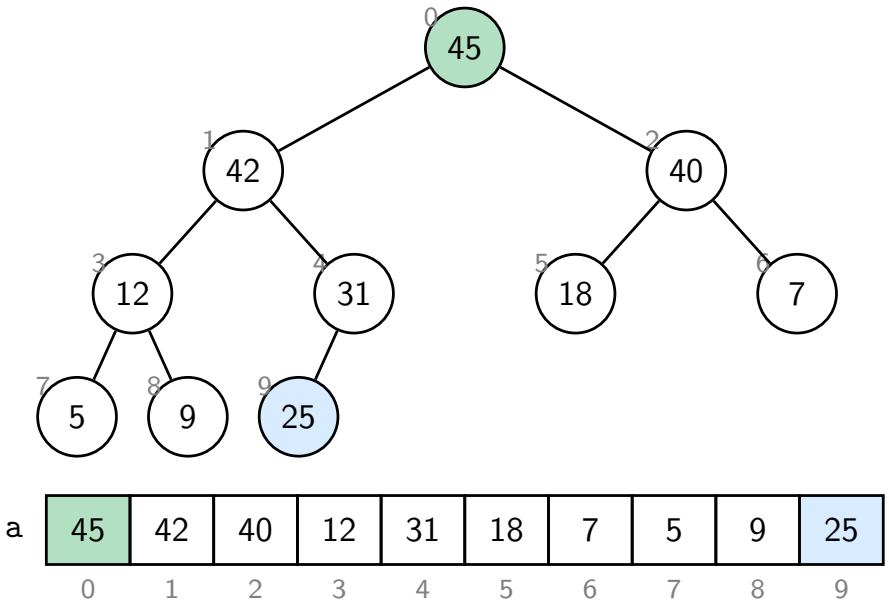
Quien usa el montículo solo ve `insertar`, `maximo`, `extraerMax`, `vacio` y `tamano`. El `push_back` pone la clave en la primera casilla libre, así que la forma sigue completa.

Consultar el máximo



El máximo está en `a[0]`: consultarlo cuesta $\Theta(1)$, sin recorrer nada.

Extraer el máximo



Pregunta

Sale el 45. La raíz queda vacía, y el árbol tiene que seguir completo con 9 claves. ¿Qué casilla debe desaparecer? ¿Qué clave ocupa la raíz?

Extraer el máximo

Respuesta

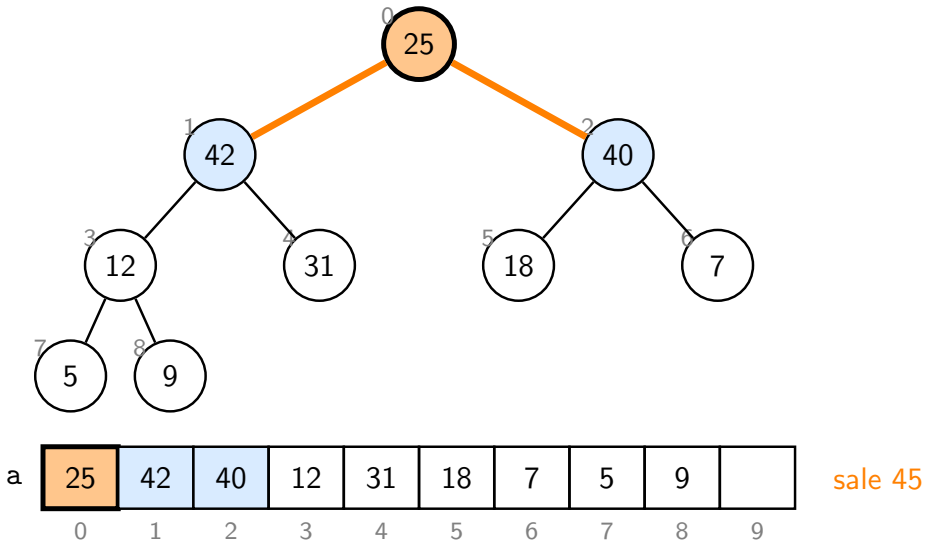
Desaparece la última casilla, `a[9]`: es la única que se puede quitar sin dejar un hueco. Su clave, el 25, pasa a la raíz. La forma queda completa, pero el 25 está encima del 42 y del 40.

Extraer el máximo: bajar

Idea (CLRS, §6.2; Thareja, §12.1.2)

La clave de la raíz se compara con sus dos hijos. Si el mayor de los hijos es mayor que ella, se intercambian y se repite desde la casilla de ese hijo. Se detiene en una hoja o cuando ningún hijo es mayor. CLRS llama a este paso `MAX-HEAPIFY`.

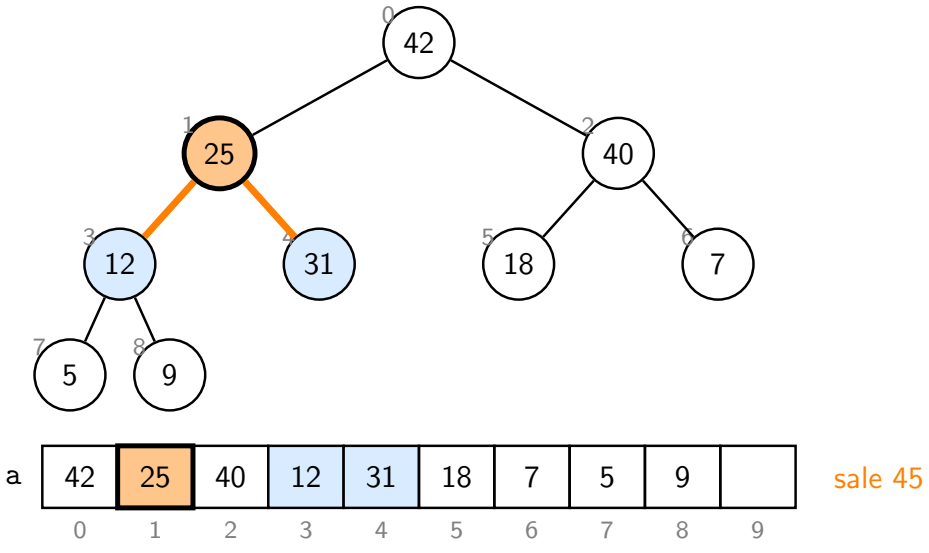
Extraer el máximo: traza



El 25 queda en `a[0]`. Sus hijos son 42 y 40; el mayor es el 42, y $42 > 25$: se intercambian.

i	hijos	comparación	acción
0	42 y 40	$42 > 25$	intercambiar, $i = 1$

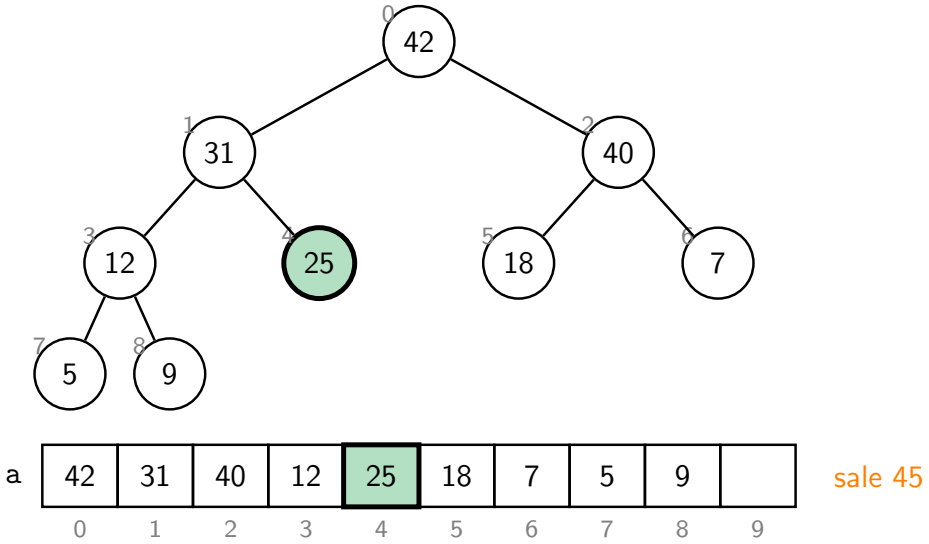
Extraer el máximo: traza



El 25 está en `a[1]`. Sus hijos son 12 y 31; el mayor es el 31, y $31 > 25$: se intercambian.

i	hijos	comparación	acción
0	42 y 40	$42 > 25$	intercambiar, $i = 1$
1	12 y 31	$31 > 25$	intercambiar, $i = 4$

Extraer el máximo: traza



El 25 está en a[4]. Sus hijos serían 9 y 10, fuera del arreglo: es una hoja y se detiene.

i	hijos	comparación	acción
0	42 y 40	$42 > 25$	intercambiar, $i = 1$
1	12 y 31	$31 > 25$	intercambiar, $i = 4$
4	fuera		hoja, detenerse

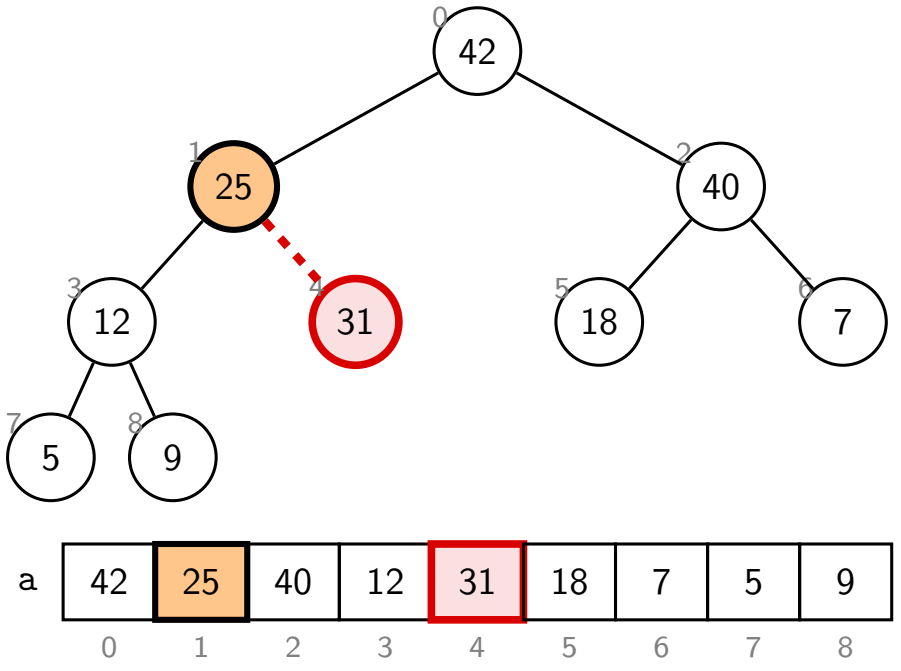
Bajar mirando un solo hijo

Pregunta

Esta versión baja comparando solo con el hijo izquierdo. Sobre 25 42 40 12 31 18 7 5 9, ¿qué arreglo deja?

```cpp
// Error: baja mirando solo el hijo izquierdo.
void bajarSoloIzq(vector<int> &a, int i) {
  int n = (int) a.size();
  while (2 * i + 1 < n && a[2 * i + 1] > a[i]) {
    int aux = a[i];
    a[i] = a[2 * i + 1];
    a[2 * i + 1] = aux;
    i = 2 * i + 1;
  }
}
```

Bajar mirando un solo hijo



En la raíz acierta por suerte: el izquierdo, 42, es el mayor. En `a[1]` el izquierdo es 12, menor que 25, y se detiene, pero el derecho es 31. Queda `42 25 40 12 31 18 7 5 9`, con el 31 bajo el 25: ya no es un montículo.

Bajar hacia el primer hijo mayor

Pregunta

Otra versión mira los dos hijos, pero intercambia con el *primero* que sea mayor que la clave, no con el mayor de los dos. ¿Qué pasa con 25 40 42?

Bajar hacia el primer hijo mayor

Respuesta

El 40 es el primero mayor que 25 y sube: queda 40 25 42, con el 42 bajo el 40. Hay que subir el mayor de los dos hijos, porque queda como padre del otro.

Extraer el máximo en C++

```
// Baja la clave de i, cambiandola por el mayor de sus hijos.
void bajar(int i) {
  int n = (int) a.size();
  bool sigue = true;
  while (sigue) {
    int izq = 2 * i + 1;
    int der = 2 * i + 2;
    int mayor = i;
    if (izq < n && a[izq] > a[mayor]) {
      mayor = izq;
    }
    if (der < n && a[der] > a[mayor]) {
      mayor = der;
    }
    if (mayor != i) {
      intercambiar(i, mayor);
      i = mayor;
    } else {
      sigue = false;
    }
  }
}
```

Extraer el máximo en C++

- En `bajar`, `mayor` empieza en `i` y se queda con la casilla de la clave más grande entre la de *i* y las de sus hijos que existan.
- Si `mayor` sigue siendo `i`, ningún hijo es mayor y `sigue` pasa a `false`: el ciclo termina por su condición.

Extraer el máximo en C++

```
int maximo() {
  assert(!a.empty());
  return a[0];
}

// La ultima clave pasa a la raiz y baja hasta su lugar.
int extraerMax() {
  assert(!a.empty());
  int res = a[0];
  a[0] = a[a.size() - 1];
  a.pop_back();
  if (!a.empty()) {
    bajar(0);
  }
  return res;
}
```

Con una sola clave, pop_back deja el vector vacío y no hay nada que bajar.

El montículo en uso

En `uso_monticulo.cpp`, `pedidos` atiende las órdenes del problema, con 0 para `atender`.

```
// Cada pedido trae su urgencia; 0 es atender el mas urgente.
void pedidos() {
  Monticulo m;
  int ordenes[9] = {15, 40, 27, 8, 0, 27, 0, 0, 0};
  int i = 0;
  while (i < 9) {
    if (ordenes[i] == 0) {
      printf("atender: %d\n", m.extraerMax());
    } else {
      m.insertar(ordenes[i]);
    }
    i = i + 1;
  }
  printf("pendientes (%d):", m.tamano());
  m.imprimir();
}
```

El montículo en uso

Antes de los pedidos, el programa inserta `45 31 40 12 25 18 7 5 9`, repite las dos trazas y extrae todo.

```
inicial:           45 31 40 12 25 18 7 5 9
insertar(42):      45 42 40 12 31 18 7 5 9 25
maximo() = 45
extraerMax() = 45: 42 31 40 12 25 18 7 5 9
salen en orden:    42 40 31 25 18 12 9 7 5
atender: 40
atender: 27
atender: 27
atender: 15
pendientes (1): 8
```

Las claves salen de mayor a menor, y los dos 27 se atienden: el montículo guarda repetidos.

Lo que cuesta cada operación del montículo

Operación	Tiempo	Por qué
`insertar(k)`	$O(\log n)$	sube a lo sumo $\lfloor \log_2 n \rfloor$ niveles, con una comparación y un intercambio por nivel; el `push_back` es $\Theta(1)$ salvo cuando el vector crece
`extraerMax()`	$O(\log n)$	baja a lo sumo $\lfloor \log_2 n \rfloor$ niveles, con dos comparaciones y un intercambio por nivel
`maximo()`	$\Theta(1)$	está en `a[0]`
`vacio()`, `tamano()`	$\Theta(1)$	el `vector` lleva la cuenta

Lo que cuesta cada operación del montículo

Operación	Tiempo	Por qué
buscar una clave cualquiera	$\Theta(n)$	entre hermanos no hay orden: no se puede descartar un subárbol
espacio de la estructura	$\Theta(n)$	solo las claves en el `vector`, sin punteros
espacio aparte	$\Theta(1)$	`subir` y `bajar` son ciclos con unas pocas variables

De vuelta a los pedidos

Estructura	`llega`	`atender`	repetidos
arreglo sin orden	$\Theta(1)$	$\Theta(n)$	sí
arreglo ordenado	$\Theta(n)$	$\Theta(1)$	sí
`set`	$O(\log n)$	$O(\log n)$	no
montículo	$O(\log n)$	$O(\log n)$	sí

Con n pedidos que llegan y se atienden, el montículo hace todo en $O(n \log n)$; el arreglo sin orden, en $\Theta(n^2)$ en el peor caso.

De vuelta a los pedidos

Estructura	espacio
arreglo sin orden	$\Theta(n)$
arreglo ordenado	$\Theta(n)$
`set`	$\Theta(n)$, con tres punteros por clave
montículo	$\Theta(n)$, solo las claves

Las cuatro ocupan $\Theta(n)$, pero el `set` guarda tres punteros por clave y el montículo solo las claves.

Contenido

1 Repaso: el árbol binario de búsqueda

2 El contenedor `set`

3 Montículos

4 Ejercicios

5 Cierre

Propuestos para la casa

1. **El más cercano.** Con un `set`, escriban la clave más cercana a x; si dos empatan, la menor.
2. **Borrar los menores.** Con un `set`, quiten de una vez todas las claves menores que x, sin un ciclo.
3. **UVa 10815, Andy's First Dictionary.** Impriman en minúscula y en orden alfabético las palabras distintas de un texto. Enunciado y envío:
   `https://vjudge.net/problem/UVA-10815`

Propuestos para la casa

4. **Extraer a mano.** Sobre el montículo `70 40 60 10 30 20 50`, extraigan el máximo tres veces y dibujen el árbol y el arreglo después de cada una.
5. **Montículo de mínimos.** Cambien `monticulo.h` para que la raíz sea el menor. ¿Qué comparaciones cambian?

Pistas para los propuestos

1. `lower_bound(x)` da la primera $\geq x$; la anterior es la otra candidata. Cuidado con `begin()` y con `end()`. $O(\log n)$.
2. `erase(desde, hasta)` con `begin()` y `lower_bound(x)`: $O(\log n + m)$.
3. Cada racha de letras, ya en minúscula, entra en un `set<string>`, que la recorre en orden alfabético.

Pistas para los propuestos

4. La primera extracción deja `60 40 50 10 30 20`.
5. Cambian las dos comparaciones de `subir` y las dos de `bajar`; `maximo` pasa a ser `minimo`.

Contenido

1 Repaso: el árbol binario de búsqueda

2 El contenedor `set`

3 Montículos

4 Ejercicios

5 Cierre

Resumen

- `map` y `set` guardan las claves en un ABB que la biblioteca mantiene balanceado: cuestan $O(\log n)$ con cualquier orden de llegada, y su recorrido es el inorden.
- `set` guarda claves distintas en orden, y `lower_bound` y `upper_bound` encuentran la primera clave $\geq x$ o $> x$ en $O(\log n)$.

Resumen

- Un montículo de máximos es un árbol binario completo en un arreglo, con cada nodo mayor o igual que sus hijos. El padre de i es $\lfloor (i-1)/2 \rfloor$ y sus hijos, $2i+1$ y $2i+2$.
- Insertar pone la clave en la primera casilla libre y la sube; extraer el máximo pasa la última clave a la raíz y la baja hacia el mayor de sus hijos. Las dos cuestan $O(\log n)$.

Lo que sigue

Después de los montículos

Construir un montículo a partir de un arreglo cualquiera en $\Theta(n)$, ordenar con un montículo (*heapsort*), y la cola de prioridad con el contenedor `priority_queue` (CLRS, §6.3 a §6.5).

Referencias



T. H. Cormen, C. E. Leiserson, R. L. Rivest y C. Stein. *Introduction to Algorithms*, 4.ª ed. MIT Press, 2022. Capítulo 6, *Heapsort*: §6.1, montículos; §6.2, mantener la propiedad de montículo; §6.5, colas de prioridad. Capítulo 12, árboles binarios de búsqueda.



R. Thareja. *Data Structures Using C*, 2.ª ed. Oxford University Press, 2014. Capítulo 12, *Heaps*: §12.1, *Binary Heaps*, p. 361; §12.1.1, insertar, p. 362; §12.1.2, borrar, p. 364.



UVa Online Judge. Problema 10815, *Andy's First Dictionary*: `https://vjudge.net/problem/UVA-10815`.